Riesgo de los commits del kernel de Linux bajo deriva temporal: un clasificador difuso-evolutivo interpretable: estado del arte, datos y modelos

Juan Felipe Rodríguez Galindo
Maestría en Ciencias de la Información y las Comunicaciones
Universidad Distrital Francisco José de Caldas, Bogotá, Colombia
Código 20261595004 · Inteligencia Computacional · Entregable 2

***Resumen*—Predecir si un commit tendrá que corregirse después permite enfocar la revisión de código. En el kernel de Linux la etiqueta es explícita: el trailer `Fixes:` de un commit posterior señala al culpable. Se plantean el problema, la pregunta y el criterio de éxito del proyecto, y se presenta una revisión estructurada tipo PRISMA en arXiv y OpenAlex con 58 estudios incluidos (37 citados aquí; la matriz completa va en el anexo, 49 desde 2021). La predicción justo a tiempo (JIT) se valida casi siempre con un único corte o con validación cruzada aleatoria y se mide con AUC-ROC, F1 o exactitud. A nivel de commit, lo difuso y lo evolutivo solo aparecen como piezas auxiliares (agrupamiento o ajuste de parámetros); como clasificador, solo a nivel de módulo. Se construye además el dataset: 612.811 commits de 2019–2026 minados del clon completo del kernel, con prevalencia de 3,9 %–4,5 %, y se justifican los cinco modelos que se comparan. Ningún trabajo evalúa un clasificador de reglas difusas a nivel de commit con validación temporal y AUC-PR, ni su degradación en 2026, el año en que el kernel pidió y luego retiró la etiqueta `Assisted-by` para contribuciones asistidas por IA.**

***Abstract*—We frame a project on just-in-time risk prediction for Linux kernel commits, labelled by later `Fixes:` trailers, and report a PRISMA-style review (arXiv and OpenAlex, 58 included studies, 37 cited here; the full matrix is in the appendix). No prior work evaluates an evolutionarily tuned fuzzy rule classifier at commit level with temporal validation and AUC-PR, nor its degradation on 2026 commits.**

***Palabras clave*—predicción de defectos justo a tiempo, kernel de Linux, deriva temporal, sistemas difusos, computación evolutiva, AUC-PR.**

I. Introducción

Desde hace dos décadas, el riesgo de un commit se estima con métricas del propio cambio: cuánto toca, cuánto se difunde y la experiencia del autor [1], [2]. Es la predicción justo a tiempo (JIT): la alerta llega cuando el cambio aún está en revisión. El kernel de Linux cuenta con una ventaja clave: cuando un commit corrige un fallo previo, el trailer `Fixes:` identifica explícitamente al culpable sin recurrir a heurísticas como SZZ mediante `git blame` [3]. En estudios empíricos, el 81 % se confirmó como causa del bug; el resto es ambiguo o incorrecto [4].

En 2026 cambió quién escribe parte de estos cambios. La versión 7.0 pidió declarar con `Assisted-by:` la ayuda de asistentes de IA [5], simplificada luego a `Assisted-by: LLM` [6]. El 14 de octubre se retiró al constatar que solo cerca del 10 % de los commits la incluía, pese a un uso real mucho mayor [7]. Por ello, la «era de la IA» se define por frontera temporal (Fig. 1) y no por la etiqueta, que queda solo como referencia complementaria.

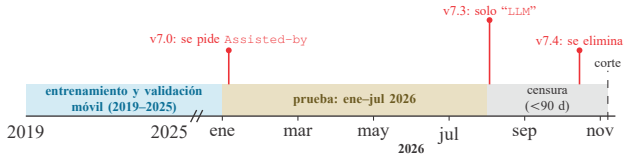


Figura 1. Diseño temporal propuesto frente a los hitos de `Assisted-by` [5]–[7]. Los commits posteriores a julio de 2026 quedan censurados: no han tenido los 90 días para recibir un `Fixes:`.

I-A. Problema y pregunta de investigación

Problema. Clasificar cada commit del kernel como *riesgoso* si un commit posterior lo cita con `Fixes:` en 90 días o menos, usando solo información disponible en el momento de integrarlo, y con un modelo que un mantenedor pueda leer.

Pregunta. ¿Un clasificador de a lo sumo 15 reglas difusas, cuyas funciones de pertenencia y reglas se ajustan con un algoritmo evolutivo, alcanza en los commits de 2026 un AUC-PR de al menos el 90 % del mejor modelo de referencia (regresión logística o *gradient boosting*), y pierde menos AUC-PR que ese modelo al pasar de la validación (ventanas anuales 2023, 2024 y 2025, entrenando desde 2019) a la prueba (enero–julio de 2026)?

I-B. Criterio de éxito (etapa 1 de CRISP-ML(Q) [8])

Sea AP_m^V el AUC-PR medio del modelo m en las ventanas de validación y AP_m^T su AUC-PR en la prueba de 2026, ambos con intervalo de confianza del 95 % por *bootstrap*. Sean $\Delta_m = (\mathrm{AP}_m^V - \mathrm{AP}_m^T)/\mathrm{AP}_m^V$ la caída relativa, π^T la prevalencia de positivos en la prueba (el AUC-PR de la clase mayoritaria), FE el modelo propuesto y R la mejor referencia clásica en validación. El proyecto se declara exitoso si se cumplen las tres condiciones de la Tabla I. Si no se cumplen, se reporta así, sin volver a ajustar nada con la prueba.

I-C. Datos y consideraciones éticas

Los datos son públicos: el historial oficial de `torvalds/linux` (GPL-2.0), minado directamente para

Tabla I
CRITERIO DE ÉXITO, FIJADO ANTES DE VER LOS DATOS

	Condición
C1 Desempeño	$AP^{T}_{FE} \geq 0{,}90\, AP^{T}_{R}$
C2 Robustez	$\Delta_{FE} < \Delta_{R}$ (diferencia observada, con IC 95 %)
C3 Piso	El IC 95 % de AP^{T}_{FE} excluye π^{T}; McNemar contra cada referencia con $\alpha = 0{,}05$
Restricción	≤15 reglas y ≤3 antecedentes por regla

un registro continuo de 2014 a 2026. Para contrastar la etiqueta se recurre además al dataset de Maes Bermejo *et al.* en Zenodo [9]. Por ser fuentes enteramente abiertas no se requiere aprobación de un comité de ética; por privacidad, los nombres y correos de los autores se sustituyen por identificadores anónimos.

II. ESTADO DEL ARTE

II-A. Método de búsqueda

La búsqueda sigue las directrices de PRISMA 2020 [10] en versión sintetizada. El problema se descompone en tres componentes temáticos esenciales, unidos con OR por dentro y AND entre ellos:

A (commit* OR "code change*" OR "software change*" OR patch* OR "pull request*" OR "Linux kernel") AND
B ("defect prediction" OR "bug prediction" OR "bug-introducing" OR "bug-inducing" OR "fix-inducing" OR "just-in-time" OR "change risk" OR "risk prediction" OR "defect-prone" OR buggy) AND
C (fuzzy OR "rule-based" OR evolutionary OR "genetic algorithm" OR "genetic programming" OR "search-based" OR interpretab* OR explainab*)

La ecuación Q1 combina los tres bloques. La Q2 cambia el bloque C por deriva temporal ("concept drift", drift, "temporal validation", "time-aware", "realistic evaluation", "data shift", "distribution shift", "time travel"). Ambas consultas se ejecutaron sobre 2019–2026 vía APIs públicas en arXiv y OpenAlex (título y resumen) para garantizar reproducibilidad. OpenAlex indexa IEEE, ACM, Springer y Elsevier, supliendo la falta de acceso institucional a Scopus o IEEE Xplore. Clásicos previos a 2019 o hallados por citas entran como "otras fuentes".

Los criterios se definieron antes del tamizaje. Se incluye si: (I1) predice el riesgo de un cambio o estudia la validez de esa evaluación (orden temporal, latencia, ruido), (I2) reporta métricas cuantitativas, y (I3) describe sus datos. Se excluye si: (E1) trata generación o reparación de código, (E2) evalúa solo a nivel de archivo o módulo (se admitieron propuestas difusas y evolutivas de módulo ante la ausencia a nivel de commit), (E3) es de otro dominio, (E4) es solo un dataset sin modelo, o (E5) no se puede verificar. De las bases se evaluaron 47 registros en elegibilidad (10 excluidos y 37 incluidos); a estos se sumaron 21 de otras fuentes bajo los mismos criterios I y E, totalizando 58 estudios incluidos (37 citados aquí; la matriz completa va en el anexo). De ellos, 35 se leyeron a texto completo y 23 por resumen o fuentes secundarias debido a restricciones de acceso institucional. El flujo se resume en la Fig. 2.

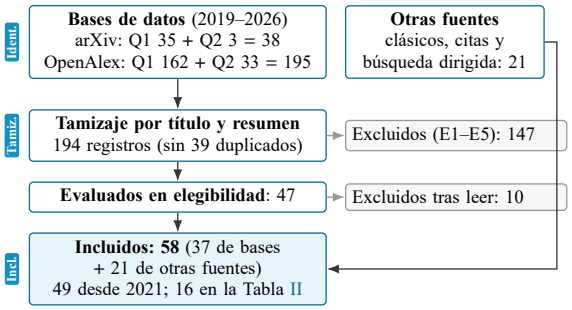


Figura 2. Flujo tipo PRISMA 2020 de la búsqueda. Conteos generados desde los registros de la búsqueda.

II-B. Síntesis por enfoques

A. Métricas de cambio y aprendizaje clásico. La línea fundacional usa métricas del commit con regresión logística [1], [2]. Sigue siendo un punto de referencia muy sólido: con partición cronológica estricta, una regresión logística de una sola variable (líneas añadidas) iguala o supera en AUC a arquitecturas profundas como DeepJIT y CC2Vec [11].

B. Aprendizaje profundo y LLM. Redes sobre el diff y modelos preentrenados con métricas [12] alcanzan techos de AUC-ROC cercanos a 0,88 en JIT-Defects4J. Tienen tres limitaciones clave: son cajas negras opacas, se evalúan casi exclusivamente en Java y rara vez reportan AUC-PR. En la industria, aunque se han usado LLMs para puntuar diffs, sus mapas de atención generaron baja confianza en los desarrolladores [13].

C. Interpretabilidad, difuso y evolutivo. Los profesionales piden explicaciones claras para cada alerta [25]. La respuesta habitual son explicadores *post hoc*: LIME en JITLine [14], reglas locales en PyExplainer [15] o commits afines en IRJIT [26], los cuales carecen de garantías formales de consistencia [27]. A nivel de commit, lo evolutivo y difuso se reducen a tareas accesorias: evolución diferencial para SMOTE [14], NSGA-II para calibrar parámetros logísticos [28], optimización de hiperparámetros [29], [30] o Fuzzy C-Means para modular capas de atención [31]. A nivel de módulo existen modelos neuro-difusos sobre datos NASA con particiones aleatorias y exactitud como métrica [16], [17] (donde una exactitud de 91 % enmascara una sensibilidad de apenas 0,33 [16]). No se identificó ningún clasificador de reglas difusas para riesgo de commits.

D. Evaluación realista, deriva y latencia. Los modelos JIT se degradan sustancialmente en el tiempo. En Qt y OpenStack, el AUC cae entre 11 y 34 puntos en solo un año sin reentrenar [18]. La validación cruzada infla los resultados: JITBoost cae de 0,891 a 0,863 de AUC al evaluar en partición temporal [32]. La deriva en JIT se aborda mediante detección continua [33], reciclaje de modelos [34], actualización en línea continua [20] o ventanas de espera [35]. Sin embargo, casi todos reportan AUC-ROC y omiten AUC-PR. La etiqueta llega tarde (latencia de 15 a más de 400 días [19]), lo que distorsiona las métricas hasta un 25 % si no se depura la cola temporal [36]. En el kernel de Linux este efecto es drástico: al pasar de datos

Tabla II

MATRIZ DEL ESTADO DEL ARTE (16 DE LOS 58 INCLUIDOS; LA MATRIZ COMPLETA VA EN EL ANEXO). ENFOQUES: A MÉTRICAS Y APRENDIZAJE CLÁSICO, B PROFUNDO Y LLM, C INTERPRETABLE, DIFUSO O EVOLUTIVO, D EVALUACIÓN REALISTA Y DERIVA, E ETIQUETA. †SOLO NIVEL DE MÓDULO. ‡PREPRINT.

Referencia	Problema y datos	Paradigma y modelo	Referencias comparadas	Validación	Métrica y resultado	Limitaciones	Relevancia para el proyecto
A Kamei [2] (2013)	11 proyectos (6 libres, 5 comerciales), >250 mil cambios; SZZ	Reg. logística, 14 métricas de cambio	Azar	VC aleatoria 10 pliegues	Precisión 34 %, sensib. 64 %; AUC 0,73 (comerciales)	VC mezcla pasado y futuro	Variables base y referencia logística
B Zeng [11] (2021)	310 mil commits, 6 proyectos (Qt, OpenStack, Go…)	Réplica de DeepJIT y CC2Vec; LApredict (RL con una variable)	DeepJIT, CC2Vec, DBN-JIT, LR-JIT	Cronológica 80/20; recorta la cola	AUC media: LApredict 0,725; DeepJIT 0,707	Una corrida; solo AUC	Lo simple iguala a lo profundo con partición temporal
B Ni [12] (2022)	JIT-Defects4J: 27 319 commits Java, 8,5 % positivos	JIT-Fine: CodeBERT + 14 métricas	LApredict, DeepJIT, CC2Vec, JITLine	Temporal 80/20 por proyecto	F1 0,431; AUC 0,881 (JITLine 0,261/0,802)	Solo Java; costo no reportado	Cota alta de AUC; modelo opaco
B Liu [13] (2026)	Diffs de Meta; cambios que causaron caídas (≈2 por mil)	LLM (Diff Risk Score) + atención por bloque	Ninguna cuantitativa	No descrita	Top-2 bloques cubren 53,9 % de líneas causantes	Una empresa; 48 de 55 opiniones negativas	Explicación post hoc frente a reglas intrínsecas
C JITLine [14] (2021)	Qt y OpenStack, 37 524 commits	RF + tokens; SMOTE ajustado por evolución diferencial; LIME	EALR, DeepJIT, CC2Vec	Una partición temporal	AUC 0,83/0,82; F1 0,33/0,24	Una partición, sin pruebas estadísticas	Usa evolución y explica, pero sobre caja negra
C PyExplainer [15] (2021)	Qt y OpenStack, 40 798 commits	Reglas locales (RuleFit) sobre RF y RL	LIME	Temporal 70/30	RF: AUC 0,75/0,74; reglas 18–38 % más fieles que LIME	Dos proyectos; vecinos aleatorios	Reglas post hoc, no un modelo de reglas
C Aftab [16] (2023)†	NASA, 5 conjuntos, 3 579 módulos	Inferencia difusa (8 reglas) que combina NB, RNA y AD	NB, RNA, AD	Aleatoria 70/30	Exactitud 91,05 %; sensib. 0,33	Reglas a mano; exactitud con desbalance	Muestra por qué no usar exactitud
C Dey [17] (2025)†	NASA: JM1, KC1, PC1, KC2, CM1	Selección multiobjetivo (estrella de mar) + integral difusa de Choquet	7 clasificadores; MOGA, MOGWO	VC aleatoria 10 pliegues	Exactitud 0,85–0,96; sin AUC	Nivel módulo; F1 y sensib. casi perfectas con 7–20 % positivos	Lo difuso-evolutivo reciente no llega al commit
D McIntosh [18] (2018)	Qt y OpenStack, 37 524 cambios (8 y 13 %)	RL no lineal con splines	Periodo corto frente a largo	Longitudinal en el tiempo	AUC cae 11–34 puntos en un año; reentrenar recupera 6–24	Dos sistemas; filtros de SZZ	Evidencia de deriva; diseño a replicar
D Cabral [19] (2019)	10 proyectos GitHub, 2003–2018	En línea: ORB (árboles de Hoeffding)	OOB, UOB, ventana deslizante	Precuencial con espera de 90 días	G-mean de JIT fijo hasta 97,2 % menor que OOB	G-mean, sin AUC-PR	Sustenta la ventana de 90 días
D Olewicki [20] (2024)	Ubisoft: 4 proyectos de riesgo, 50–82 mil commits	MLP con aprendizaje continuo	XGBoost reentrenado, ingenuo	Secuencial en el tiempo	F1 +9 a +31 % en 6 de 8 proyectos	Una empresa; sin latencia	Deriva industrial en riesgo de commits
D Nguyen [21] (2025)	Linux (993 704 commits) y FFmpeg; Linux con `Fixes:`	8 técnicas JIT de vulnerabilidades (RL, DeepJIT, JIT-Fine…)	Escenario idealizado frente a realista	Temporal 75/5/20	PR-AUC en Linux 0,805 → 0,016 (−98 %)	Vulnerabilidades, no defectos	El más cercano: Linux, `Fixes:`, AUC-PR
D Mazloom-zadeh [22] (2026)‡	AIDev: 489 repos. Python, 9 428 PR de agentes	Estudio longitudinal (GLMM, SZZ)	PR humanos	Por trimestres	Sin diferencia relevante en inducción de bugs (V = 0,068)	SZZ con P=0,39	La era IA cambia las covariables
E Lyu [4] (2024)	KFC: 76 046 commits con `Fixes:`, Linux 2013–2022	6 variantes de SZZ + TC-SZZ	Rosa et al.	Contra `Fixes:` como verdad	R-SZZ F1 0,51; 81 % de `Fixes:` confirmados	`Fixes:` es opcional	Valida la etiqueta del proyecto
E Herbold [23] (2022)	398 versiones, 38 proyectos Apache	Calidad de datos; NB entre proyectos	SZZ a 6 meses frente a etiqueta validada	Entre proyectos; Friedman + Nemenyi	Cerca de la mitad de las correcciones de SZZ no lo son	Solo Apache	Ruido de SZZ; preferir `Fixes:`
E Falessi [24] (2022)	19 proyectos Apache, >4 000 defectos	15 clasificadores de Weka	Con y sin defectos latentes	Por versiones, 66/33	Pérdida mediana de sensib. ≈90 % por defectos latentes	Nivel de clase	Censura: recortar la cola reciente

balanceados artificialmente a la prevalencia real, el AUC-PR de ocho métodos cae de 0,805 a 0,016 [21]. Además, los commits de agentes no elevan la tasa de defectos, pero alteran el tamaño y estructura de los cambios [22], ubicando la deriva en las covariables.

E. Fiabilidad de la etiqueta. SZZ sufre de alto ruido: casi la mitad de los commits que señala como correctores no lo son [23], y el filtrado con LLM reduce 21 % los casos sospechosos en PostgreSQL [37]. En Linux, variantes de SZZ alcanzan un F1 de apenas 0,49–0,51 frente a la verdad del trailer `Fixes:` [4]. Defectos latentes aún no descubiertos bajan la sensibilidad cerca de un 90 % si no se aplica una ventana prudencial de censura temporal [24].

II-C. Brecha

Los enfoques anteriores dejan cuatro vacíos concretos:

1) Inexistencia de un clasificador de reglas difusas optimizado mediante algoritmos evolutivos para predecir riesgo a nivel de commit.
2) Escasa medición de la deriva temporal con AUC-PR y prevalencia real en commits del kernel respaldados por trailers `Fixes:`.
3) Falta de estudios sobre si la interpretabilidad intrínseca penaliza el desempeño o aporta robustez frente a la deriva.
4) 2026 es el primer año en que el kernel pidió declarar la ayuda de IA [5], aunque la etiqueta se retiró en octubre por su bajo uso [7].

III. METODOLOGÍA: DATOS Y MODELOS

La metodología sigue las etapas de CRISP-ML(Q) [8]. Esta sección cubre la comprensión y preparación de los datos (etapa 2) y la elección de los modelos (etapa 3). El protocolo de evaluación completo se presenta en la Entrega 3.

III-A. Construcción del dataset

Se minó un clon completo de `torvalds/linux` con una sola pasada de `git log` sobre la rama principal, sin commits de fusión, desde 2014. Los años 2014–2018 solo sirven para calcular la experiencia de los autores. Por cada commit se extraen el *numstat* del diff, los trailers del mensaje y las fechas. Las referencias `Fixes:` (prefijos de al menos 8 caracteres hexadecimales) se resuelven contra el historial: el 9,4 % que no se resuelve apunta a commits anteriores a 2014 o a otros árboles.

Etiqueta. Un commit es *riesgoso* si algún commit posterior de la rama principal lo cita con `Fixes:` en 90 días o menos. Los commits con menos de 90 días de observación a la fecha de corte quedan *censurados* y se excluyen (26.133 commits), porque su etiqueta negativa todavía no es definitiva [19], [24].

Tabla III
FICHA DEL DATASET Y VERIFICACIÓN D1–D7

	Valor
Fuente	Clon de `torvalds/linux`, rama principal, GPL-2.0
Periodo	2019-01-01 a 2026-07-31
D1 Tamaño	612.811 commits no censurados: 558.438 de desarrollo (2019–2025) y 54.373 de prueba (2026)
D2 Objetivo	Riesgoso (binaria): 23.735 positivos en desarrollo, prevalencia anual entre 3,9 % y 4,5 %
D3 Calidad	Sin faltantes ni hashes duplicados; colas pesadas (hasta 730 092 líneas en un commit); 3 fechas de autor corruptas
D4 Estructura	Eventos irregulares ordenados por fecha del committer; 18.127 autores y 748 subsistemas
D5 Disponibilidad	Solo información del propio commit y del pasado del autor
D6 Licencia y ética	Metadatos públicos; autores anonimizados
D7 Documentación	Diccionario de las 22 variables en el anexo

Anonimización. Nombre y correo del autor se reemplazan por un SHA-256 con sal. No se publica el texto de los mensajes.

La Tabla III resume la ficha del dataset y su validación con los criterios D1–D7 del enunciado. Las 22 variables candidatas son las familias clásicas de JIT [2]:

- tamaño: líneas añadidas y borradas, archivos y binarios;
- difusión: directorios de primer nivel y subsistemas;
- revisión: `Reviewed-by`, `Acked-by`, `Tested-by`, `Reported-by`, `Signed-off-by`, `Co-developed-by` y `Link`;
- propósito: si trae su propio `Fixes:`, si pide backport a *stable*, si el asunto menciona *fix* y la longitud del asunto;
- experiencia: commits previos del autor, en total y en el subsistema, y días desde su primer commit;
- momento: hora y día de la semana.

Todas se conocen al integrar el commit.

III-B. Análisis exploratorio

Hay tres hallazgos que guían el diseño:

- **La prevalencia depende de la adopción de `Fixes:`.** Antes de 2019 crece año a año (Fig. 3), porque cada vez más correcciones usan el trailer, no porque el código empeore. Desde 2019 se estabiliza entre 3,9 % y 4,5 %, y por eso el entrenamiento empieza el 1 de enero de 2019. En 2026 baja a 3,39 %.
- **El tamaño es la señal dominante y no es lineal.** La prevalencia pasa de 0,9 % en el quintil más pequeño de líneas añadidas a 11,7 % en el más grande, con una distribución de colas muy pesadas.
- **Hay deriva en las covariables antes de 2026.** El número medio de `Reviewed-by` por commit sube de 0,42 (2019) a 0,71 (2025), y la fracción de commits que son correcciones, de 14,5 % a 18,4 %.

En correlación de Spearman, dos pares son casi redundantes: directorios de primer nivel y subsistemas (0,97), y experiencia y antigüedad del autor (0,73). Por eso el modelo difuso usa solo uno de cada par.

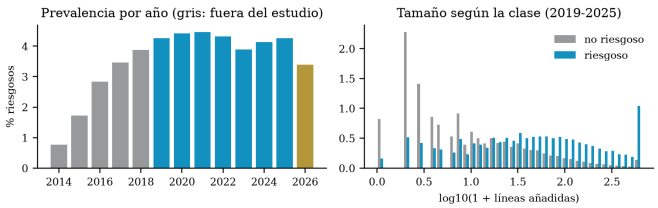


Figura 3. Izquierda: prevalencia de commits riesgosos por año (en gris, los años fuera del estudio). Derecha: distribución del tamaño por clase en 2019–2025.

Tabla IV
MODELOS COMPARADOS

Modelo	Papel	Justificación
Clase mayoritaria	Trivial	Su AUC-PR es la prevalencia: el piso
Reg. logística	Clásica	Estándar de JIT [2]; $\log(1+x)$ y estandarización
Gradient boosting	Clásica fuerte	Cota alta sin restricción de interpretabilidad [38]
Difuso-evolutivo	Propuesto	$\leq$15 reglas legibles; vértices y reglas ajustados con un AG
Difuso sin evolución	Variante de control	Mismas reglas iniciales, sin AG: mide el aporte de la evolución

III-C. Modelos y su papel

La Tabla IV resume los modelos. Las dos referencias clásicas cubren dos extremos. La regresión logística es el estándar de JIT y es difícil de superar con partición temporal [2], [11]. El *gradient boosting* [38] da la cota alta de una caja negra sobre las mismas variables.

Qué propiedad de los datos aprovecha el modelo propuesto. El riesgo crece por umbrales con el tamaño y la difusión, se modula con la revisión y la experiencia, y las variables tienen colas muy pesadas. Una partición difusa sobre la escala de cuantiles captura esos umbrales sin depender de la escala. Las reglas con pocos antecedentes capturan las interacciones que un mantenedor reconoce, por ejemplo "si el cambio es grande y toca varios subsistemas". El modelo usa 10 variables fijadas antes de ver resultados: líneas añadidas y borradas, archivos, subsistemas, `Reviewed-by`, `Signed-off-by`, si es corrección, si va a *stable* y la experiencia del autor, en total y en el subsistema.

III-D. Clasificador difuso-evolutivo

Cada variable x_j se lleva a $z_j \in [0,1]$ con su función de distribución empírica, estimada solo con el entrenamiento. Sobre z_j se definen tres términos (bajo, medio, alto) que forman una partición de Ruspini [39] con vértices $a_j < b_j < c_j$: bajo vale 1 hasta a_j y cae a 0 en b_j; medio es el triángulo (a_j, b_j, c_j); alto sube de 0 en b_j a 1 en c_j. Una regla r tiene hasta tres antecedentes y un peso $w_r \in [0,1]$:

$$\mu_r(\mathbf{x}) = \prod_{(j,t)\in A_r} \mu_{jt}(z_j), \qquad s(\mathbf{x}) = 1 - \prod_r \big(1 - w_r\,\mu_r(\mathbf{x})\big), \tag{1}$$

es decir, t-norma producto y O probabilístico entre reglas. El puntaje s ordena los commits por riesgo.

El ajuste sigue el enfoque Pittsburgh de los sistemas difusos genéticos [40], [41]: cada individuo codifica todos los vértices y la base completa de reglas (activación, antecedentes, términos y pesos). La aptitud es el AUC-PR de entrenamiento, que es la métrica adecuada con clases desbalanceadas [42], menos una penalización por el número de antecedentes. Los operadores son:

- selección por torneo y elitismo;
- cruce uniforme, que hereda cada regla completa;
- mutación gaussiana de vértices y pesos, y mutación aleatoria de antecedentes y términos.

La variante sin evolución fija los vértices en los cuantiles (0,2; 0,5; 0,8) y elige reglas de 1 o 2 antecedentes de forma voraz, por su confianza difusa, mientras mejore el AUC-PR. Esa misma solución siembra la población inicial del AG. Así, la diferencia entre ambas variantes aísla el aporte de la evolución.

Declaración de uso de IA generativa

Usé Gemini para revisar y corregir errores y para redactar este informe a partir de los resultados del cuaderno y la recuperación de algunos papers para la evaluación del mismo.

Referencias

[1] A. Mockus and D. M. Weiss, "Predicting risk of software changes," Bell Labs Tech. J., vol. 5, no. 2, pp. 169–180, Apr.–Jun. 2000, doi: 10.1002/bltj.2229.

[2] Y. Kamei, E. Shihab, B. Adams, A. E. Hassan, A. Mockus, A. Sinha, and N. Ubayashi, "A large-scale empirical study of just-in-time quality assurance," IEEE Trans. Softw. Eng., vol. 39, no. 6, pp. 757–773, Jun. 2013, doi: 10.1109/TSE.2012.70.

[3] J. Śliwerski, T. Zimmermann, and A. Zeller, "When do changes induce fixes?" in Proc. Int. Workshop Mining Softw. Repositories (MSR '05), St. Louis, MO, USA, 2005, pp. 1–5, doi: 10.1145/1083142.1083147.

[4] Y. Lyu, H. J. Kang, R. Widyasari, J. Lawall, and D. Lo, "Evaluating SZZ implementations: An empirical study on the Linux kernel," IEEE Trans. Softw. Eng., vol. 50, no. 9, pp. 2219–2239, Sep. 2024, doi: 10.1109/TSE.2024.3406718.

[5] S. Levin, "docs: add AI Coding Assistants documentation," Linux kernel, commit 78d979db6cef, Jan. 6, 2026. [Online]. Available: https://github.com/torvalds/linux/commit/78d979db6cef

[6] C. Brauner, "coding-assistants: simplify attribution," Linux kernel, commit 816d9992d9ed, Aug. 3, 2026. [Online]. Available: https://github.com/torvalds/linux/commit/816d9992d9ed

[7] S. Levin, "docs: remove Assisted-by references," Linux kernel, commit 1e035297ffda, Oct. 14, 2026. [Online]. Available: https://github.com/torvalds/linux/commit/1e035297ffda

[8] S. Studer, T. B. Bui, C. Drescher, A. Hanuschkin, L. Winkler, S. Peters, and K.-R. Müller, "Towards CRISP-ML(Q): A machine learning process model with quality assurance methodology," Mach. Learn. Knowl. Extr., vol. 3, no. 2, pp. 392–413, 2021, doi: 10.3390/make3020020.

[9] M. Maes Bermejo, J. M. González-Barahona, M. Gallego, and G. Robles, "A dataset of Linux kernel commits" (v3), Zenodo, 2024, doi: 10.5281/zenodo.10654193.

[10] M. J. Page *et al.*, "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," BMJ, vol. 372, Art. no. n71, 2021, doi: 10.1136/bmj.n71.

[11] Z. Zeng, Y. Zhang, H. Zhang, and L. Zhang, "Deep just-in-time defect prediction: How far are we?" in Proc. 30th ACM SIGSOFT Int. Symp. Softw. Testing Anal. (ISSTA), Virtual, Denmark, 2021, pp. 427–438, doi: 10.1145/3460319.3464819.

[12] C. Ni, W. Wang, K. Yang, X. Xia, K. Liu, and D. Lo, "The best of both worlds: Integrating semantic features with expert features for defect prediction and localization," in Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), Singapore, 2022, pp. 672–683, doi: 10.1145/3540250.3549165.

[13] Y. Liu, K. Jabre, R. Abreu, Z. J. Carmichael, V. Murali, A. Patel, J. Ge, W. Sun, C. Zhang, A. Mockus, D. Khavari, P. C. Rigby, and N. Nagappan, "A preliminary study on explaining risk of code changes using LLM-based prediction models," in Proc. 3rd ACM Int. Conf. AI-Powered Softw. (AIware '26), Montreal, QC, Canada, 2026, pp. 259–267, doi: 10.1145/3805760.3814916.

[14] C. Pornprasit and C. K. Tantithamthavorn, "JITLine: A simpler, better, faster, finer-grained just-in-time defect prediction," in Proc. IEEE/ACM 18th Int. Conf. Mining Softw. Repositories (MSR), 2021, pp. 369–379, doi: 10.1109/MSR52588.2021.00049.

[15] C. Pornprasit, C. Tantithamthavorn, J. Jiarpakdee, M. Fu, and P. Thongtanunam, "PyExplainer: Explaining the predictions of just-in-time defect models," in Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE), 2021, pp. 407–418, doi: 10.1109/ASE51524.2021.9678763.

[16] S. Aftab, S. Abbas, T. M. Ghazal, M. Ahmad, H. A. Hamadi, C. Y. Yeun, and M. A. Khan, "A cloud-based software defect prediction system using data and decision-level machine learning fusion," Mathematics, vol. 11, no. 3, Art. no. 632, 2023, doi: 10.3390/math11030632.

[17] R. Dey, J. Piri, B. Acharya, P. P. Das, V. C. Gerogiannis, and A. Kanavos, "A hybrid evolutionary fuzzy ensemble approach for accurate software defect prediction," Mathematics, vol. 13, no. 7, Art. no. 1140, 2025, doi: 10.3390/math13071140.

[18] S. McIntosh and Y. Kamei, "Are fix-inducing changes a moving target? A longitudinal case study of just-in-time defect prediction," IEEE Trans. Softw. Eng., vol. 44, no. 5, pp. 412–428, May 2018, doi: 10.1109/TSE.2017.2693980.

[19] G. G. Cabral, L. L. Minku, E. Shihab, and S. Mujahid, "Class imbalance evolution and verification latency in just-in-time software defect prediction," in Proc. IEEE/ACM 41st Int. Conf. Softw. Eng. (ICSE), Montreal, QC, Canada, 2019, pp. 666–676, doi: 10.1109/ICSE.2019.00076.

[20] D. Olewicki, S. Habchi, M. Nayrolles, M. Faramarzi, S. Chandar, and B. Adams, "On the costs and benefits of adopting lifelong learning for software analytics – Empirical study on brown build and risk prediction," in Proc. 46th Int. Conf. Softw. Eng.: Softw. Eng. Pract. (ICSE-SEIP), Lisbon, Portugal, 2024, pp. 275–286, doi: 10.1145/3639477.3639717.

[21] D. Nguyen, T. Le-Cong, T. H. M. Le, M. A. Babar, and Q.-T. Huynh, "Toward realistic evaluations of just-in-time vulnerability prediction," in Proc. IEEE Int. Conf. Softw. Maint. Evol. (ICSME), Auckland, New Zealand, 2025, pp. 1–10, doi: 10.1109/ICSME64153.2025.00023.

[22] I. Mazloomzadeh, M. M. Morovati, and F. Khomh, "How do AI coding agents contribute to software development? An empirical study of agentic pull requests," arXiv:2607.21832, Jul. 2026, doi: 10.48550/arXiv.2607.21832.

[23] S. Herbold, A. Trautsch, F. Trautsch, and B. Ledel, "Problems with SZZ and features: An empirical study of the state of practice of defect prediction data collection," Empir. Softw. Eng., vol. 27, no. 2, Art. no. 42, 2022, doi: 10.1007/s10664-021-10092-4.

[24] D. Falessi, A. Ahluwalia, and M. Di Penta, "The impact of dormant defects on defect prediction: A study of 19 Apache projects," ACM Trans. Softw. Eng. Methodol., vol. 31, no. 1, pp. 1–26, Jan. 2022, doi: 10.1145/3467895.

[25] J. Jiarpakdee, C. K. Tantithamthavorn, and J. Grundy, "Practitioners' perceptions of the goals and visual explanations of defect prediction models," in Proc. IEEE/ACM 18th Int. Conf. Mining Softw. Repositories (MSR), 2021, pp. 432–443, doi: 10.1109/MSR52588.2021.00055.

[26] H. Sahar, A. A. Bangash, A. Hindle, and D. Barbosa, "IRJIT: A simple, online, information retrieval approach for just-in-time software defect prediction," Empir. Softw. Eng., vol. 29, no. 5, Art. no. 131, 2024, doi: 10.1007/s10664-024-10514-z.

[27] J. Yu, M. Fu, A. Ignatiev, C. Tantithamthavorn, and P. Stuckey, "A formal explainer for just-in-time defect predictions," ACM Trans. Softw. Eng. Methodol., vol. 33, no. 7, pp. 1–31, Sep. 2024, doi: 10.1145/3664809.

[28] X. Chen, Y. Zhao, Q. Wang, and Z. Yuan, "MULTI: Multi-objective effort-aware just-in-time software defect prediction," Inf. Softw. Technol., vol. 93, pp. 1–13, Jan. 2018, doi: 10.1016/j.infsof.2017.08.004.

[29] J. Kang, S. Kwon, D. Ryu, and J. Baik, "HASPO: Harmony search-based parameter optimization for just-in-time software defect prediction

in maritime software," Appl. Sci., vol. 11, no. 5, Art. no. 2002, 2021, doi: 10.3390/app11052002.
[30] S. Mostafa, S. T. Cynthia, B. Roy, and D. Mondal, "Feature transformation for improved software bug detection and commit classification," J. Syst. Softw., vol. 219, Art. no. 112205, Jan. 2025, doi: 10.1016/j.jss.2024.112205.
[31] F. Chen, Y. Ke, X. Liu, and Q. Wei, "Multi-project just-in-time software defect prediction based on multi-task learning for mobile applications," in Proc. IEEE Conf. Softw. Testing, Verification Validation (ICST), 2025, pp. 127–137, doi: 10.1109/ICST62969.2025.10989004.
[32] M. A. Shehab, A. Hamou-Lhadj, and V. S. P. Gunda, "JITBoost: Boosting just-in-time defect prediction using Boolean combination of classifiers," in Proc. IEEE 23rd Int. Conf. Softw. Qual., Rel. Secur. (QRS), 2023, pp. 95–104, doi: 10.1109/QRS60937.2023.00019.
[33] Z. Chitsazian, S. Sedighian Kashi, and A. Nikanjam, "Detecting concept drift in just-in-time software defect prediction using model interpretation," Int. J. Syst. Assur. Eng. Manag., vol. 17, no. 9, pp. 2701–2722, 2026, doi: 10.1007/s13198-025-03122-7.
[34] H. J. Patel, B. Adams, and A. E. Hassan, "Post deployment recycling of machine learning models," Empir. Softw. Eng., vol. 29, no. 4, Art. no. 100, 2024, doi: 10.1007/s10664-024-10492-2.
[35] D. Tian, L. Song, and X. Su, "Just-in-time quality assurance with assistance of automated defect prediction and code testing," ACM Trans. Softw. Eng. Methodol., vol. 35, no. 9, pp. 1–46, 2026, doi: 10.1145/3779653.
[36] L. Song and L. L. Minku, "A procedure to continuously evaluate predictive performance of just-in-time software defect prediction models during software development," IEEE Trans. Softw. Eng., vol. 49, no. 2, pp. 646–666, Feb. 2023, doi: 10.1109/TSE.2022.3158831.
[37] R. C. Veras, G. G. Cabral, and A. L. I. de Oliveira, "Semantic SZZ: Mitigating the impact of misclassified corrective changes in just-in-time software defect prediction," in Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC), 2025, pp. 769–774, doi: 10.1109/SMC58881.2025.11342537.
[38] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189–1232, Oct. 2001, doi: 10.1214/aos/1013203451.
[39] E. H. Ruspini, "A new approach to clustering," Inf. Control, vol. 15, no. 1, pp. 22–32, Jul. 1969, doi: 10.1016/S0019-9958(69)90591-9.
[40] F. Herrera, "Genetic fuzzy systems: taxonomy, current research trends and prospects," Evol. Intell., vol. 1, no. 1, pp. 27–46, Mar. 2008, doi: 10.1007/s12065-007-0001-5.
[41] H. Ishibuchi, T. Nakashima, and T. Murata, "Three-objective genetics-based machine learning for linguistic rule extraction," Inf. Sci., vol. 136, no. 1–4, pp. 109–133, Aug. 2001, doi: 10.1016/S0020-0255(01)00144-X.
[42] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLOS ONE, vol. 10, no. 3, Art. no. e0118432, Mar. 2015, doi: 10.1371/journal.pone.0118432.